DÉMO 005-B · CHANGEMENTS ENTRE LES ÉTAPES

Redux puis Redux Toolkit

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-basic (4 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-multi-reducers (2 fichiers)
3. **Étape 2 → 3** · etape-3-provider (6 fichiers)
4. **Étape 3 → 4** · etape-4-payload (5 fichiers)
5. **Étape 4 → 5** · etape-5-middleware (6 fichiers)
6. **Étape 5 → 6** · etape-6-premier-slice (10 fichiers)
7. **Étape 6 → 7** · etape-7-redux-toolkit (10 fichiers)

Étape 1 etape-1-basic : point de départ

createStore et un seul reducer : BUY_PHONE, BUY_TV, BUY_CAMERA ; getState, dispatch, subscribe

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (67 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : react-redux ^9.2.0, redux ^5.0.1

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Home from "./pages/Home.jsx";

function App() {
  return <Home />;
}

export default App;
```

DÉPART **src/pages/Home.jsx** 60 lignes

```jsx
import phone from "../assets/phone.png";
import tv from "../assets/tv.png";
import camera from "../assets/camera.png";

import { buyPhone, buyTV, buyCamera, store } from "../store/index.js";
function Home() {
  // getState lit le state une fois : l'écran ne se met pas à jour tout seul (d'où le subscribe)
  const state = store.getState();
  return (
    <div>
```

```
      <h2>{"Welcome to our Application"}</h2>
      <div className="container">
        <div className="item">
          <img src={phone} alt="" width={100} height={100} />
          <div className="availability" id="phones">
            {state.phones} available
          </div>
          <button
            className="button"
            onClick={() => {
              store.dispatch(buyPhone());
            }}
          >
            Buy
          </button>
        </div>
        <div className="item">
          <img src={tv} alt="" width={100} height={100} />
          <div className="availability" id="tvs">
            {state.tvs} available
          </div>
          <button
            className="button"
            onClick={() => {
              store.dispatch(buyTV());
            }}
          >
            Buy
          </button>
        </div>
        <div className="item">
          <img src={camera} alt="" width={100} height={100} />
          <div className="availability" id="cameras">
            {state.cameras} available
          </div>
          <button
            className="button"
            onClick={() => {
              store.dispatch(buyCamera());
            }}
          >
            Buy
          </button>
        </div>
      </div>
    </div>
  );
}

export default Home;
```

DÉPART **src/store/index.js** 73 lignes

```
import { createStore } from "redux";

// Action Types

const BUY_PHONE = "BUY_PHONE";
const BUY_TV = "BUY_TV";
const BUY_CAMERA = "BUY_CAMERA";

const buyPhone = () => {
  return {
    type: BUY_PHONE,
  };
```

```
};

const buyTV = () => {
  return {
    type: BUY_TV,
  };
};

const buyCamera = () => {
  return {
    type: BUY_CAMERA,
  };
};

// reducer

// initial state
const initialState = {
  phones: 5,
  tvs: 3,
  cameras: 8,
};

// (previousState, action) => newState
const phoneReducer = (state = initialState, action) => {
  switch (action.type) {
    case BUY_PHONE:
      return {
        ...state,
        phones: state.phones - 1,
      };
    case BUY_TV:
      return {
        ...state,
        tvs: state.tvs - 1,
      };
    case BUY_CAMERA:
      return {
        ...state,
        cameras: state.cameras - 1,
      };
    default:
      return state;
  }
};

// store
// createStore : le store garde tout le state ; le reducer dit comment il change
const store = createStore(phoneReducer);

// subscribe : cette fonction est appelée après chaque dispatch
store.subscribe(() => {
  const state = store.getState();
  console.log("Updated State:", state);
  // Update the UI elements directly (Test)
  document.getElementById("phones").innerHTML = `${state.phones} available`;
  document.getElementById("tvs").innerHTML = `${state.tvs} available`;
  document.getElementById("cameras").innerHTML = `${state.cameras} available`;
});

export { store, buyPhone, buyTV, buyCamera, phoneReducer };
```

Étape 1 → 2 etape-1-basic → etape-2-multi-reducers

Un reducer par produit, combinés avec combineReducers

MODIFIÉ `src/pages/Home.jsx` +11 −10

```
@@ -1,19 +1,20 @@
- import phone from "../assets/phone.png";
- import tv from "../assets/tv.png";
- import camera from "../assets/camera.png";
-
+ import phoneImg from "../assets/phone.png";
+ import tvImg from "../assets/tv.png";
+ import cameraImg from "../assets/camera.png";
  import { buyPhone, buyTV, buyCamera, store } from "../store/index.js";
  function Home() {
    // getState lit le state une fois : l'écran ne se met pas à jour tout seul (d'où le subscribe)
    const state = store.getState();
+   const { phone, tv, camera } = state;
+
    return (
      <div>
        <h2>{"Welcome to our Application"}</h2>
        <div className="container">
          <div className="item">
-           <img src={phone} alt="" width={100} height={100} />
+           <img src={phoneImg} alt="" width={100} height={100} />
            <div className="availability" id="phones">
-             {state.phones} available
+             {phone.total} available
            </div>
            <button
              className="button"
@@ -25,9 +26,9 @@
            </button>
          </div>
          <div className="item">
-           <img src={tv} alt="" width={100} height={100} />
+           <img src={tvImg} alt="" width={100} height={100} />
            <div className="availability" id="tvs">
-             {state.tvs} available
+             {tv.total} available
            </div>
            <button
              className="button"
@@ -39,9 +40,9 @@
            </button>
          </div>
          <div className="item">
-           <img src={camera} alt="" width={100} height={100} />
+           <img src={cameraImg} alt="" width={100} height={100} />
            <div className="availability" id="cameras">
-             {state.cameras} available
+             {camera.total} available
            </div>
            <button
              className="button"
```

MODIFIÉ `src/store/index.js` +51 −23

```
@@ -1,4 +1,4 @@
- import { createStore } from "redux";
```

```
+ import { combineReducers, createStore } from "redux";
  // Action Types
@@ -27,47 +27,75 @@
  // reducer
  // initial state
- const initialState = {
-   phones: 5,
-   tvs: 3,
-   cameras: 8,
+ const initialStatePhone = {
+   total: 5,
+ };
+
+ const initialStateTV = {
+   total: 3,
+ };
+
+ const initialStateCamera = {
+   total: 8,
  };

  // (previousState, action) => newState
- const phoneReducer = (state = initialState, action) => {
+ const phoneReducer = (state = initialStatePhone, action) => {
    switch (action.type) {
      case BUY_PHONE:
        return {
          ...state,
-         phones: state.phones - 1,
        };
-     case BUY_TV:
-       return {
-         ...state,
-         tvs: state.tvs - 1,
-       };
-     case BUY_CAMERA:
-       return {
-         ...state,
-         cameras: state.cameras - 1,
+         total: state.total - 1,
        };
      default:
        return state;
    }
  };

- // store
- // createStore : le store garde tout le state ; le reducer dit comment il change
- const store = createStore(phoneReducer);
+ // (previousState, action) => newState
+ const tvsReducer = (state = initialStateTV, action) => {
+   switch (action.type) {
+     case BUY_TV:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
```

```
+ // (previousState, action) => newState
+ const camerasReducer = (state = initialStateCamera, action) => {
+   switch (action.type) {
+     case BUY_CAMERA:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
+ const rootReducer = combineReducers({
+   phone: phoneReducer,
+   tv: tvsReducer,
+   camera: camerasReducer,
+ });
+
+ const store = createStore(rootReducer);

  // subscribe : cette fonction est appelée après chaque dispatch
  store.subscribe(() => {
    const state = store.getState();
+   const { phone, tv, camera } = state;
    console.log("Updated State:", state);
    // Update the UI elements directly (Test)
-   document.getElementById("phones").innerHTML = `${state.phones} available`;
-   document.getElementById("tvs").innerHTML = `${state.tvs} available`;
-   document.getElementById("cameras").innerHTML = `${state.cameras} available`;
+   document.getElementById("phones").innerHTML = `${phone.total} available`;
+   document.getElementById("tvs").innerHTML = `${tv.total} available`;
+   document.getElementById("cameras").innerHTML = `${camera.total} available`;
  });

  export { store, buyPhone, buyTV, buyCamera, phoneReducer };
```

Étape 2 → 3 etape-2-multi-reducers → etape-3-provider

Dossier store/ (actions, reducers, store), <Provider>, useSelector, useDispatch

MODIFIÉ `src/App.jsx` +8 −1

```
@@ -1,8 +1,15 @@
  import "./App.css";
  import Home from "./pages/Home.jsx";
+ import { Provider } from "react-redux";
+ import { store } from "./store/store.js";

  function App() {
-   return <Home />;
+   // Provider donne le store à tous les composants (useSelector, useDispatch)
+   return (
+     <Provider store={store}>
+       <Home />
+     </Provider>
+   );
  }

  export default App;
```

MODIFIÉ `src/pages/Home.jsx` +13 −7

```
@@ -1,11 +1,17 @@
  import phoneImg from "../assets/phone.png";
  import tvImg from "../assets/tv.png";
  import cameraImg from "../assets/camera.png";
- import { buyPhone, buyTV, buyCamera, store } from "../store/index.js";
+
+ import { store } from "../store/store";
+ import { buyPhone, buyTV, buyCamera } from "../store/actions";
+ import { useSelector, useDispatch } from "react-redux";
+

  function Home() {
-   // getState lit le state une fois : l'écran ne se met pas à jour tout seul (d'où le subscribe)
-   const state = store.getState();
-   const { phone, tv, camera } = state;
+   // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
+   const dispatch = useDispatch();
+   const phone = useSelector((state) => state.phone);
+   const camera = useSelector((state) => state.camera);
+   const tv = useSelector((state) => state.tv);

    return (
      <div>
@@ -19,7 +25,7 @@
              <button
                className="button"
                onClick={() => {
-                 store.dispatch(buyPhone());
+                 dispatch(buyPhone());
                }}
              >
                Buy
@@ -33,7 +39,7 @@
              <button
                className="button"
                onClick={() => {
-                 store.dispatch(buyTV());
```

```
                dispatch(buyTV());
              }}
            >
              Buy
@@ -47,7 +53,7 @@
            <button
              className="button"
              onClick={() => {
-               store.dispatch(buyCamera());
+               dispatch(buyCamera());
              }}
            >
              Buy
```

AJOUTÉ **src/store/actions.js** +25 −0

```
@@ -1,0 +1,25 @@
+ // Action Types
+
+ const BUY_PHONE = "BUY_PHONE";
+ const BUY_TV = "BUY_TV";
+ const BUY_CAMERA = "BUY_CAMERA";
+
+ const buyPhone = () => {
+   return {
+     type: BUY_PHONE,
+   };
+ };
+
+ const buyTV = () => {
+   return {
+     type: BUY_TV,
+   };
+ };
+
+ const buyCamera = () => {
+   return {
+     type: BUY_CAMERA,
+   };
+ };
+
+ export { BUY_PHONE, BUY_TV, BUY_CAMERA, buyPhone, buyTV, buyCamera };
```

SUPPRIMÉ **src/store/index.js** +0 −101

```
@@ -1,101 +1,0 @@
- import { combineReducers, createStore } from "redux";
-
- // Action Types
-
- const BUY_PHONE = "BUY_PHONE";
- const BUY_TV = "BUY_TV";
- const BUY_CAMERA = "BUY_CAMERA";
-
- const buyPhone = () => {
-   return {
-     type: BUY_PHONE,
-   };
- };
-
- const buyTV = () => {
-   return {
-     type: BUY_TV,
-   };
```

```
- };
-
- const buyCamera = () => {
-   return {
-     type: BUY_CAMERA,
-   };
- };
-
- // reducer
-
- // initial state
- const initialStatePhone = {
-   total: 5,
- };
-
- const initialStateTV = {
-   total: 3,
- };
-
- const initialStateCamera = {
-   total: 8,
- };
-
- // (previousState, action) => newState
- const phoneReducer = (state = initialStatePhone, action) => {
-   switch (action.type) {
-     case BUY_PHONE:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
-     default:
-       return state;
-   }
- };
-
- // (previousState, action) => newState
- const tvsReducer = (state = initialStateTV, action) => {
-   switch (action.type) {
-     case BUY_TV:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
-     default:
-       return state;
-   }
- };
-
- // (previousState, action) => newState
- const camerasReducer = (state = initialStateCamera, action) => {
-   switch (action.type) {
-     case BUY_CAMERA:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
-     default:
-       return state;
-   }
- };
-
- // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.cam
era)
- const rootReducer = combineReducers({
```

```
-   phone: phoneReducer,
-   tv: tvsReducer,
-   camera: camerasReducer,
- });
-
- const store = createStore(rootReducer);
-
- // subscribe : cette fonction est appelée après chaque dispatch
- store.subscribe(() => {
-   const state = store.getState();
-   const { phone, tv, camera } = state;
-   console.log("Updated State:", state);
-   // Update the UI elements directly (Test)
-   document.getElementById("phones").innerHTML = `${phone.total} available`;
-   document.getElementById("tvs").innerHTML = `${tv.total} available`;
-   document.getElementById("cameras").innerHTML = `${camera.total} available`;
- });
-
- export { store, buyPhone, buyTV, buyCamera, phoneReducer };
```

AJOUTÉ **src/store/reducers.js** +57 −0

```
@@ -1,0 +1,57 @@
+ // reducer
+
+ import { BUY_CAMERA, BUY_PHONE, BUY_TV } from "./actions";
+
+ // initial state
+ const initialStatePhone = {
+   total: 5,
+ };
+
+ const initialStateTV = {
+   total: 3,
+ };
+
+ const initialStateCamera = {
+   total: 8,
+ };
+
+ // (previousState, action) => newState
+ const phoneReducer = (state = initialStatePhone, action) => {
+   switch (action.type) {
+     case BUY_PHONE:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ // (previousState, action) => newState
+ const tvsReducer = (state = initialStateTV, action) => {
+   switch (action.type) {
+     case BUY_TV:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
```

```
+
+ // (previousState, action) => newState
+ const camerasReducer = (state = initialStateCamera, action) => {
+   switch (action.type) {
+     case BUY_CAMERA:
+       return {
+         ...state,
+         total: state.total - 1,
+       };
+     default:
+       return state;
+   }
+ };
+
+ export { phoneReducer, tvsReducer, camerasReducer };
```

AJOUTÉ **src/store/store.js** +21 −0

```
@@ -1,0 +1,21 @@
+ import { combineReducers, createStore, applyMiddleware } from "redux";
+ import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
+
+ // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
+ const rootReducer = combineReducers({
+   phone: phoneReducer,
+   tv: tvsReducer,
+   camera: camerasReducer,
+ });
+
+ // Simple logging middleware
+ const loggerMiddleware = (store) => (next) => (action) => {
+   console.log("Action:", action);
+   const result = next(action);
+   console.log("State after action:", store.getState());
+   return result;
+ };
+
+ const store = createStore(rootReducer, applyMiddleware(loggerMiddleware));
+
+ export { store };
```

Étape 3 → 4 etape-3-provider → etape-4-payload

Action avec payload : choisir la quantité de téléphones

MODIFIÉ `src/App.css` +13 −0

```
@@ -65,3 +65,16 @@
    font-weight: 600;
    color: white;
  }
+
+ .quantity {
+   margin: 0;
+ }
+
+ .input {
+   width: 60px;
+   padding: 4px;
+   font-size: 16px;
+   border: 1px solid #ccc;
+   border-radius: 4px;
+   text-align: center;
+ }
```

MODIFIÉ `src/pages/Home.jsx` +39 −2

```
@@ -2,16 +2,23 @@
  import tvImg from "../assets/tv.png";
  import cameraImg from "../assets/camera.png";

- import { store } from "../store/store";
  import { buyPhone, buyTV, buyCamera } from "../store/actions";
  import { useSelector, useDispatch } from "react-redux";
+ import { useState } from "react";

  function Home() {
+   const [quantity, setQuantity] = useState(1);
+
    // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
    const dispatch = useDispatch();
    const phone = useSelector((state) => state.phone);
    const camera = useSelector((state) => state.camera);
    const tv = useSelector((state) => state.tv);
+
+   const handleChange = (e) => {
+     const value = e.target.value;
+     setQuantity(Number(value)); // un input donne toujours du texte : on le met en nombre
+   };

    return (
      <div>
@@ -22,10 +29,20 @@
              <div className="availability" id="phones">
                {phone.total} available
              </div>
+             <div className="quantity">
+               <input
+                 className="input"
+                 type="number"
+                 min="1"
+                 max={phone.total}
+                 defaultValue={1}
+                 onChange={handleChange}
```

```
                />
              </div>
              <button
                className="button"
                onClick={() => {
-                 dispatch(buyPhone());
+                 dispatch(buyPhone(quantity));
                }}
              >
                Buy
@@ -35,6 +52,16 @@
              <img src={tvImg} alt="" width={100} height={100} />
              <div className="availability" id="tvs">
                {tv.total} available
              </div>
+             <div className="quantity">
+               <input
+                 className="input"
+                 type="number"
+                 min="1"
+                 max={tv.total}
+                 defaultValue={1}
+                 disabled
+               />
              </div>
              <button
                className="button"
@@ -49,6 +76,16 @@
              <img src={cameraImg} alt="" width={100} height={100} />
              <div className="availability" id="cameras">
                {camera.total} available
              </div>
+             <div className="quantity">
+               <input
+                 className="input"
+                 type="number"
+                 min="1"
+                 max={camera.total}
+                 defaultValue={1}
+                 disabled
+               />
              </div>
              <button
                className="button"
```

MODIFIÉ **src/store/actions.js** +3 −1

```
@@ -4,9 +4,11 @@
  const BUY_TV = "BUY_TV";
  const BUY_CAMERA = "BUY_CAMERA";

- const buyPhone = () => {
+ const buyPhone = (quantity) => {
    return {
      type: BUY_PHONE,
+     // payload : la donnée envoyée avec l'action (ici la quantité choisie)
+     payload: quantity,
    };
  };
```

MODIFIÉ **src/store/reducers.js** +1 −1

```
@@ -21,7 +21,7 @@
```

```
21  21        case BUY_PHONE:
22  22          return {
23  23            ...state,
24      -         total: state.total - 1,
    24  +         total: state.total - action.payload,
25  25          };
26  26        default:
27  27          return state;
```

MODIFIÉ **src/store/store.js** +6 −9

```
    @@ -1,4 +1,4 @@
1       - import { combineReducers, createStore, applyMiddleware } from "redux";
    1   + import { combineReducers, createStore } from "redux";
2   2     import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
3   3
4   4     // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
    @@ -8,14 +8,11 @@
8   8       camera: camerasReducer,
9   9     });
10  10
11      - // Simple logging middleware
12      - const loggerMiddleware = (store) => (next) => (action) => {
13      -   console.log("Action:", action);
14      -   const result = next(action);
15      -   console.log("State after action:", store.getState());
16      -   return result;
17      - };
    11  + const store = createStore(rootReducer);
18  12
19      - const store = createStore(rootReducer, applyMiddleware(loggerMiddleware));
    13  + // subscribe : cette fonction est appelée après chaque dispatch
    14  + store.subscribe(() => {
    15  +   console.log("Updated State:", store.getState());
    16  + });
20  17
21  18    export { store };
```

Étape 4 → 5 etape-4-payload → etape-5-middleware

Middleware : redux-thunk + axios pour charger les commentaires (loading / success / error)

MODIFIÉ **package.json** +3 −1

```
@@ -10,10 +10,12 @@
    "preview": "vite preview"
  },
  "dependencies": {
+   "axios": "^1.13.2",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-redux": "^9.2.0",
-   "redux": "^5.0.1"
+   "redux": "^5.0.1",
+   "redux-thunk": "^3.1.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
```

MODIFIÉ **src/App.css** +35 −0

```
@@ -78,3 +78,38 @@
    border-radius: 4px;
    text-align: center;
  }
+
+ .page {
+   box-sizing: border-box;
+   place-items: center;
+ }
+
+ .comments-section {
+   padding: 12px;
+   width: 610px;
+   border: 0.5px solid mediumslateblue;
+   background-color: beige;
+   box-sizing: border-box;
+   border-radius: 4px;
+ }
+
+ .comment {
+   padding: 12px;
+   border: 1px solid #eee;
+   border-radius: 4px;
+   margin-bottom: 12px;
+   background-color: lightyellow;
+   border: none;
+ }
+
+ .comment-name {
+   font-weight: 600;
+   margin-bottom: 8px;
+   text-align: left;
+ }
+
+ .comment-body {
+   font-weight: 400;
+   white-space: pre-wrap;
+   text-align: left;
+ }
```

MODIFIÉ **src/pages/Home.jsx** +103 −70

```
@@ -4,7 +4,8 @@
4  4
5  5   import { buyPhone, buyTV, buyCamera } from "../store/actions";
6  6   import { useSelector, useDispatch } from "react-redux";
7     - import { useState } from "react";
   7   + import { useEffect, useState } from "react";
   8   + import { apiCall } from "../store/comments/actions";
8  9
9  10  function Home() {
10 11    const [quantity, setQuantity] = useState(1);
@@ -14,87 +15,119 @@
14 15    const phone = useSelector((state) => state.phone);
15 16    const camera = useSelector((state) => state.camera);
16 17    const tv = useSelector((state) => state.tv);
   18  +   const comments = useSelector((state) => state.comments);
17 19
18 20    const handleChange = (e) => {
19 21      const value = e.target.value;
20 22      setQuantity(Number(value)); // un input donne toujours du texte : on le met en nombre
21 23    };
22 24
   25  +   useEffect(() => {
   26  +     // Utiliser l'action thunk pour charger les commentaires
   27  +     dispatch(apiCall());
   28  +   }, []);
   29  +
23 30    return (
24 31      <div>
25 32        <h2>{"Welcome to our Application"}</h2>
26    -       <div className="container">
27    -         <div className="item">
28    -           <img src={phoneImg} alt="" width={100} height={100} />
29    -           <div className="availability" id="phones">
30    -             {phone.total} available
   33  +       <div className="page">
   34  +         <div className="container">
   35  +           <div className="item">
   36  +             <img src={phoneImg} alt="" width={100} height={100} />
   37  +             <div className="availability" id="phones">
   38  +               {phone.total} available
   39  +             </div>
   40  +             <div className="quantity">
   41  +               <input
   42  +                 className="input"
   43  +                 type="number"
   44  +                 min="1"
   45  +                 max={phone.total}
   46  +                 defaultValue={1}
   47  +                 onChange={handleChange}
   48  +               />
   49  +             </div>
   50  +             <button
   51  +               className="button"
   52  +               onClick={() => {
   53  +                 dispatch(buyPhone(quantity));
   54  +               }}
   55  +             >
   56  +               Buy
   57  +             </button>
31 58            </div>
32    -           <div className="quantity">
33    -             <input
```

```
34        -                 className="input"
35        -                 type="number"
36        -                 min="1"
37        -                 max={phone.total}
38        -                 defaultValue={1}
39        -                 onChange={handleChange}
40        -               />
      59  +             <div className="item">
      60  +               <img src={tvImg} alt="" width={100} height={100} />
      61  +               <div className="availability" id="tvs">
      62  +                 {tv.total} available
      63  +               </div>
      64  +               <div className="quantity">
      65  +                 <input
      66  +                   className="input"
      67  +                   type="number"
      68  +                   min="1"
      69  +                   max={tv.total}
      70  +                   defaultValue={1}
      71  +                   disabled
      72  +                 />
      73  +               </div>
      74  +               <button
      75  +                 className="button"
      76  +                 onClick={() => {
      77  +                   dispatch(buyTV());
      78  +                 }}
      79  +               >
      80  +                 Buy
      81  +               </button>
41    82                 </div>
42        -               <button
43        -                 className="button"
44        -                 onClick={() => {
45        -                   dispatch(buyPhone(quantity));
46        -                 }}
47        -               >
48        -                 Buy
49        -               </button>
      83  +               <div className="item">
      84  +                 <img src={cameraImg} alt="" width={100} height={100} />
      85  +                 <div className="availability" id="cameras">
      86  +                   {camera.total} available
      87  +                 </div>
      88  +                 <div className="quantity">
      89  +                   <input
      90  +                     className="input"
      91  +                     type="number"
      92  +                     min="1"
      93  +                     max={camera.total}
      94  +                     defaultValue={1}
      95  +                     disabled
      96  +                   />
      97  +                 </div>
      98  +                 <button
      99  +                   className="button"
     100  +                   onClick={() => {
     101  +                     dispatch(buyCamera());
     102  +                   }}
     103  +                 >
     104  +                   Buy
     105  +                 </button>
     106  +               </div>
50   107             </div>
51        -           <div className="item">
```

```
52      -           <img src={tvImg} alt="" width={100} height={100} />
53      -           <div className="availability" id="tvs">
54      -             {tv.total} available
55      -           </div>
56      -           <div className="quantity">
57      -             <input
58      -               className="input"
59      -               type="number"
60      -               min="1"
61      -               max={tv.total}
62      -               defaultValue={1}
63      -               disabled
64      -             />
65      -           </div>
66      -           <button
67      -             className="button"
68      -             onClick={() => {
69      -               dispatch(buyTV());
70      -             }}
71      -           >
72      -             Buy
73      -           </button>
74      -         </div>
75      -         <div className="item">
76      -           <img src={cameraImg} alt="" width={100} height={100} />
77      -           <div className="availability" id="cameras">
78      -             {camera.total} available
79      -           </div>
80      -           <div className="quantity">
81      -             <input
82      -               className="input"
83      -               type="number"
84      -               min="1"
85      -               max={camera.total}
86      -               defaultValue={1}
87      -               disabled
88      -             />
89      -           </div>
90      -           <button
91      -             className="button"
92      -             onClick={() => {
93      -               dispatch(buyCamera());
94      -             }}
95      -           >
96      -             Buy
97      -           </button>
108     +         <div className="comments-section">
109     +           {comments.isLoading ? (
110     +             <div>Chargement des commentaires...</div>
111     +           ) : comments.error ? (
112     +             <div>Erreur: {comments.error}</div>
113     +           ) : (
114     +             <div className="comments-list">
115     +               {comments.comments.length > 0 ? (
116     +                 comments.comments
117     +                   .slice(0, 10)
118     +                   .map((comment, index) => (
119     +                     <div key={comment.id || index} className="comment">
120     +                       <div className="comment-name">
121     +                         {comment.name || comment.email}
122     +                       </div>
123     +                       <div className="comment-body">{comment.body}</div>
124     +                     </div>
125     +                   ))
126     +               ) : (
```

```
127 +                 <div>Aucun commentaire disponible</div>
128 +               )}
129 +             </div>
130 +           )}
98  131           </div>
99  132         </div>
100 133       </div>
```

AJOUTÉ **src/store/comments/actions.js** +41 −0

```
@@ -1,0 +1,41 @@
1  + export const LOAD_COMMENTS = "LOAD_COMMENTS";
2  + export const LOAD_COMMENTS_SUCCESS = "LOAD_COMMENTS_SUCCESS";
3  + export const LOAD_COMMENTS_ERROR = "LOAD_COMMENTS_ERROR";
4  +
5  + import axios from "axios";
6  +
7  + const loadApiComments = () => {
8  +   return {
9  +     type: LOAD_COMMENTS,
10 +   };
11 + };
12 +
13 + const loadCommentsSuccess = (comments) => {
14 +   return {
15 +     type: LOAD_COMMENTS_SUCCESS,
16 +     payload: comments,
17 +   };
18 + };
19 +
20 + const loadCommentsError = (error) => {
21 +   return {
22 +     type: LOAD_COMMENTS_ERROR,
23 +     payload: error,
24 +   };
25 + };
26 +
27 + // Avec redux-thunk, une action peut être une fonction : elle reçoit dispatch et fait l'app
   el API
28 + export const apiCall = () => {
29 +   return async (dispatch) => {
30 +     dispatch(loadApiComments());
31 +
32 +     try {
33 +       const response = await axios.get(
34 +         "https://jsonplaceholder.typicode.com/comments?postId=1"
35 +       );
36 +       dispatch(loadCommentsSuccess(response.data));
37 +     } catch (error) {
38 +       dispatch(loadCommentsError(error.message));
39 +     }
40 +   };
41 + };
```

AJOUTÉ **src/store/comments/reducers.js** +40 −0

```
@@ -1,0 +1,40 @@
1  + import {
2  +   LOAD_COMMENTS,
3  +   LOAD_COMMENTS_ERROR,
4  +   LOAD_COMMENTS_SUCCESS,
5  + } from "./actions";
6  +
7  + const initialStateComments = {
8  +   isLoading: false,
```

```
+   comments: [],
+   error: "",
+ };
+
+ const commentReducer = (state = initialStateComments, action) => {
+   switch (action.type) {
+     case LOAD_COMMENTS:
+       return {
+         ...state,
+         isLoading: true,
+       };
+     case LOAD_COMMENTS_SUCCESS:
+       return {
+         ...state,
+         isLoading: false,
+         comments: action.payload,
+         error: "",
+       };
+     case LOAD_COMMENTS_ERROR:
+       return {
+         ...state,
+         isLoading: false,
+         comments: [],
+         error: action.payload,
+       };
+
+     default:
+       return state;
+   }
+ };
+
+ export default commentReducer;
```

MODIFIÉ **src/store/store.js** +16 −2

```
@@ -1,14 +1,28 @@
- import { combineReducers, createStore } from "redux";
+ import { applyMiddleware, combineReducers, createStore, compose } from "redux";
  import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
+ import commentReducer from "./comments/reducers";
+ // Import nommé au lieu de default
+ import { thunk } from "redux-thunk";

  // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
  const rootReducer = combineReducers({
    phone: phoneReducer,
    tv: tvsReducer,
    camera: camerasReducer,
+   comments: commentReducer,
  });

- const store = createStore(rootReducer);
+ // applyMiddleware(thunk) : permet de dispatcher des fonctions (apiCall), pas seulement des objets
+ const store = createStore(rootReducer, applyMiddleware(thunk));
+
+ /*
+ const composeEnhancers = window.__REDUX_DEVTOOLS_EXTENSION_COMPOSE__ || compose;
+
+ const store = createStore(
+   rootReducer,
+   composeEnhancers(applyMiddleware(thunk))
+ );
+ */
```

```
12  26
13  27  // subscribe : cette fonction est appelée après chaque dispatch
14  28  store.subscribe(() => {
```

Étape 5 → 6 etape-5-middleware → etape-6-premier-slice

Redux Toolkit : configureStore, un premier createSlice (phone)

MODIFIÉ `package.json` +2 −4

```
@@ -10,12 +10,10 @@
    "preview": "vite preview"
  },
  "dependencies": {
-   "axios": "^1.13.2",
+   "@reduxjs/toolkit": "^2.11.2",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
-   "react-redux": "^9.2.0",
-   "redux": "^5.0.1",
-   "redux-thunk": "^3.1.0"
+   "react-redux": "^9.2.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
```

MODIFIÉ `src/App.css` +0 −35

```
@@ -78,38 +78,3 @@
    border-radius: 4px;
    text-align: center;
  }
-
- .page {
-   box-sizing: border-box;
-   place-items: center;
- }
-
- .comments-section {
-   padding: 12px;
-   width: 610px;
-   border: 0.5px solid mediumslateblue;
-   background-color: beige;
-   box-sizing: border-box;
-   border-radius: 4px;
- }
-
- .comment {
-   padding: 12px;
-   border: 1px solid #eee;
-   border-radius: 4px;
-   margin-bottom: 12px;
-   background-color: lightyellow;
-   border: none;
- }
-
- .comment-name {
-   font-weight: 600;
-   margin-bottom: 8px;
-   text-align: left;
- }
-
- .comment-body {
-   font-weight: 400;
-   white-space: pre-wrap;
-   text-align: left;
- }
```

MODIFIÉ **src/App.jsx** +0 −1

```
@@ -2,7 +2,6 @@
  import Home from "./pages/Home.jsx";
  import { Provider } from "react-redux";
  import { store } from "./store/store.js";
-
  function App() {
    // Provider donne le store à tous les composants (useSelector, useDispatch)
    return (
```

MODIFIÉ **src/pages/Home.jsx** +25 −116

```
@@ -2,132 +2,41 @@
  import tvImg from "../assets/tv.png";
  import cameraImg from "../assets/camera.png";

- import { buyPhone, buyTV, buyCamera } from "../store/actions";
- import { useSelector, useDispatch } from "react-redux";
- import { useEffect, useState } from "react";
- import { apiCall } from "../store/comments/actions";
+ // buyPhone vient du slice ; buyTv et buyCamera encore des anciennes actions
+ import { buyPhone } from "../store/phoneSlice";
+ import { buyTv, buyCamera } from "../store/actions";
+ import { useDispatch, useSelector } from "react-redux";

  function Home() {
-   const [quantity, setQuantity] = useState(1);
-
    // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
    const dispatch = useDispatch();
    const phone = useSelector((state) => state.phone);
+   const tv = useSelector((state) => state.tv);
    const camera = useSelector((state) => state.camera);
-   const tv = useSelector((state) => state.tv);
-   const comments = useSelector((state) => state.comments);
-
-   const handleChange = (e) => {
-     const value = e.target.value;
-     setQuantity(Number(value)); // un input donne toujours du texte : on le met en nombre
-   };
-
-   useEffect(() => {
-     // Utiliser l'action thunk pour charger les commentaires
-     dispatch(apiCall());
-   }, []);

    return (
      <div>
        <h2>{"Welcome to our Application"}</h2>
-       <div className="page">
-         <div className="container">
-           <div className="item">
-             <img src={phoneImg} alt="" width={100} height={100} />
-             <div className="availability" id="phones">
-               {phone.total} available
-             </div>
-             <div className="quantity">
-               <input
-                 className="input"
-                 type="number"
-                 min="1"
-                 max={phone.total}
-                 defaultValue={1}
-                 onChange={handleChange}
```

```
48  -               />
49  -             </div>
50  -             <button
51  -               className="button"
52  -               onClick={() => {
53  -                 dispatch(buyPhone(quantity));
54  -               }}
55  -             >
56  -               Buy
57  -             </button>
58  -           </div>
59  -           <div className="item">
60  -             <img src={tvImg} alt="" width={100} height={100} />
61  -             <div className="availability" id="tvs">
62  -               {tv.total} available
63  -             </div>
64  -             <div className="quantity">
65  -               <input
66  -                 className="input"
67  -                 type="number"
68  -                 min="1"
69  -                 max={tv.total}
70  -                 defaultValue={1}
71  -                 disabled
72  -               />
73  -             </div>
74  -             <button
75  -               className="button"
76  -               onClick={() => {
77  -                 dispatch(buyTV());
78  -               }}
79  -             >
80  -               Buy
81  -             </button>
82  -           </div>
83  -           <div className="item">
84  -             <img src={cameraImg} alt="" width={100} height={100} />
85  -             <div className="availability" id="cameras">
86  -               {camera.total} available
87  -             </div>
88  -             <div className="quantity">
89  -               <input
90  -                 className="input"
91  -                 type="number"
92  -                 min="1"
93  -                 max={camera.total}
94  -                 defaultValue={1}
95  -                 disabled
96  -               />
97  -             </div>
98  -             <button
99  -               className="button"
100 -               onClick={() => {
101 -                 dispatch(buyCamera());
102 -               }}
103 -             >
104 -               Buy
105 -             </button>
106 -           </div>
19  +       <div className="container">
20  +         <div className="item">
21  +           <img src={phoneImg} alt="" width={100} height={100} />
22  +           <div className="availability">{phone.total} available</div>
23  +           <button className="button" onClick={() => dispatch(buyPhone(2))}>
24  +             Buy
```

```
                25 +           </button>
107             26           </div>
108               -         <div className="comments-section">
109               -           {comments.isLoading ? (
110               -             <div>Chargement des commentaires...</div>
111               -           ) : comments.error ? (
112               -             <div>Erreur: {comments.error}</div>
113               -           ) : (
114               -             <div className="comments-list">
115               -               {comments.comments.length > 0 ? (
116               -                 comments.comments
117               -                   .slice(0, 10)
118               -                   .map((comment, index) => (
119               -                     <div key={comment.id || index} className="comment">
120               -                       <div className="comment-name">
121               -                         {comment.name || comment.email}
122               -                       </div>
123               -                       <div className="comment-body">{comment.body}</div>
124               -                     </div>
125               -                   ))
126               -               ) : (
127               -                 <div>Aucun commentaire disponible</div>
128               -               )}
129               -             </div>
130               -           )}
                27 +         <div className="item">
                28 +           <img src={tvImg} alt="" width={100} height={100} />
                29 +           <div className="availability">{tv.total} available</div>
                30 +           <button className="button" onClick={() => dispatch(buyTv())}>
                31 +             Buy
                32 +           </button>
                33 +         </div>{" "}
                34 +         <div className="item">
                35 +           <img src={cameraImg} alt="" width={100} height={100} />
                36 +           <div className="availability">{camera.total} available</div>
                37 +           <button className="button" onClick={() => dispatch(buyCamera())}>
                38 +             Buy
                39 +           </button>
131             40           </div>
132             41         </div>
133             42       </div>
```

MODIFIÉ **src/store/actions.js** +5 −15

```
                @@ -1,22 +1,12 @@
1                 - // Action Types
                1 + const BUY_TV = "BUY_TV";
2               2
3                 - const BUY_PHONE = "BUY_PHONE";
4                 - const BUY_TV = "BUY_TV";
5                 - const BUY_CAMERA = "BUY_CAMERA";
6                 -
7                 - const buyPhone = (quantity) => {
8                 -   return {
9                 -     type: BUY_PHONE,
10                -     // payload : la donnée envoyée avec l'action (ici la quantité choisie)
11                -     payload: quantity,
12                -   };
13                - };
14                -
15                - const buyTV = () => {
                3 + const buyTv = () => {
16              4     return {
17              5       type: BUY_TV,
18              6     };
```

```
19  7    };
    8  +
    9  + const BUY_CAMERA = "BUY_CAMERA";
20  10
21  11   const buyCamera = () => {
22  12     return {
           @@ -24,4 +14,4 @@
24  14     };
25  15   };
26  16
27     - export { BUY_PHONE, BUY_TV, BUY_CAMERA, buyPhone, buyTV, buyCamera };
       17 + export { BUY_TV, buyTv, BUY_CAMERA, buyCamera };
```

SUPPRIMÉ **src/store/comments/actions.js** +0 −41

```
       @@ -1,41 +1,0 @@
1      - export const LOAD_COMMENTS = "LOAD_COMMENTS";
2      - export const LOAD_COMMENTS_SUCCESS = "LOAD_COMMENTS_SUCCESS";
3      - export const LOAD_COMMENTS_ERROR = "LOAD_COMMENTS_ERROR";
4      -
5      - import axios from "axios";
6      -
7      - const loadApiComments = () => {
8      -   return {
9      -     type: LOAD_COMMENTS,
10     -   };
11     - };
12     -
13     - const loadCommentsSuccess = (comments) => {
14     -   return {
15     -     type: LOAD_COMMENTS_SUCCESS,
16     -     payload: comments,
17     -   };
18     - };
19     -
20     - const loadCommentsError = (error) => {
21     -   return {
22     -     type: LOAD_COMMENTS_ERROR,
23     -     payload: error,
24     -   };
25     - };
26     -
27     - // Avec redux-thunk, une action peut être une fonction : elle reçoit dispatch et fait l'app
       el API
28     - export const apiCall = () => {
29     -   return async (dispatch) => {
30     -     dispatch(loadApiComments());
31     -
32     -     try {
33     -       const response = await axios.get(
34     -         "https://jsonplaceholder.typicode.com/comments?postId=1"
35     -       );
36     -       dispatch(loadCommentsSuccess(response.data));
37     -     } catch (error) {
38     -       dispatch(loadCommentsError(error.message));
39     -     }
40     -   };
41     - };
```

SUPPRIMÉ **src/store/comments/reducers.js** +0 −40

```
       @@ -1,40 +1,0 @@
1      - import {
2      -   LOAD_COMMENTS,
```

```
-   LOAD_COMMENTS_ERROR,
-   LOAD_COMMENTS_SUCCESS,
- } from "./actions";
-
- const initialStateComments = {
-   isLoading: false,
-   comments: [],
-   error: "",
- };
-
- const commentReducer = (state = initialStateComments, action) => {
-   switch (action.type) {
-     case LOAD_COMMENTS:
-       return {
-         ...state,
-         isLoading: true,
-       };
-     case LOAD_COMMENTS_SUCCESS:
-       return {
-         ...state,
-         isLoading: false,
-         comments: action.payload,
-         error: "",
-       };
-     case LOAD_COMMENTS_ERROR:
-       return {
-         ...state,
-         isLoading: false,
-         comments: [],
-         error: action.payload,
-       };
-
-     default:
-       return state;
-   }
- };
-
- export default commentReducer;
```

AJOUTÉ **src/store/phoneSlice.js** +17 −0

```
@@ -1,0 +1,17 @@
+ import { createSlice } from "@reduxjs/toolkit";
+
+ const phoneSlice = createSlice({
+   name: "phone",
+   initialState: {
+     total: 5,
+   },
+   // Chaque reducer devient une action (phoneSlice.actions) ; Immer permet de « modifier » state directement
+   reducers: {
+     buyPhone: (state, action) => {
+       state.total -= action.payload || 1;
+     },
+   },
+ });
+
+ export const { buyPhone } = phoneSlice.actions;
+ export const phoneReducer = phoneSlice.reducer;
```

MODIFIÉ **src/store/reducers.js** +10 −31

```
@@ -1,35 +1,11 @@
- // reducer
```

```
+ // phone est passé dans phoneSlice : il reste tv et camera (passés en slice à l'étape 7)
+ import { BUY_TV, BUY_CAMERA } from "./actions";
  
- import { BUY_CAMERA, BUY_PHONE, BUY_TV } from "./actions";
- 
- // initial state
- const initialStatePhone = {
-   total: 5,
- };
- 
- const initialStateTV = {
+ const tvInitialState = {
    total: 3,
  };
  
- const initialStateCamera = {
-   total: 8,
- };
- 
- // (previousState, action) => newState
- const phoneReducer = (state = initialStatePhone, action) => {
-   switch (action.type) {
-     case BUY_PHONE:
-       return {
-         ...state,
-         total: state.total - action.payload,
-       };
-     default:
-       return state;
-   }
- };
- 
- // (previousState, action) => newState
- const tvsReducer = (state = initialStateTV, action) => {
+ const tvReducer = (state = tvInitialState, action) => {
    switch (action.type) {
      case BUY_TV:
        return {
@@ -41,8 +17,11 @@
    }
  };
  
- // (previousState, action) => newState
- const camerasReducer = (state = initialStateCamera, action) => {
+ const cameraInitialState = {
+   total: 8,
+ };
+ 
+ const cameraReducer = (state = cameraInitialState, action) => {
    switch (action.type) {
      case BUY_CAMERA:
        return {
@@ -54,4 +33,4 @@
    }
  };
  
- export { phoneReducer, tvsReducer, camerasReducer };
+ export { tvReducer, cameraReducer };
```

MODIFIÉ **src/store/store.js** +11 −30

```
@@ -1,32 +1,13 @@
- import { applyMiddleware, combineReducers, createStore, compose } from "redux";
- import { camerasReducer, phoneReducer, tvsReducer } from "./reducers";
```

```
- import commentReducer from "./comments/reducers";
- // Import nommé au lieu de default
- import { thunk } from "redux-thunk";
+ import { configureStore } from "@reduxjs/toolkit";
+ import { phoneReducer } from "./phoneSlice";
+ import { tvReducer, cameraReducer } from "./reducers";

- // combineReducers : chaque reducer gère sa part du state (state.phone, state.tv, state.camera)
- const rootReducer = combineReducers({
-   phone: phoneReducer,
-   tv: tvsReducer,
-   camera: camerasReducer,
-   comments: commentReducer,
+ // configureStore remplace createStore + combineReducers : on peut mélanger un slice et les anciens reducers
+ export const store = configureStore({
+   reducer: {
+     phone: phoneReducer,
+     tv: tvReducer,
+     camera: cameraReducer,
+   },
+   devTools: true,
  });
-
- // applyMiddleware(thunk) : permet de dispatcher des fonctions (apiCall), pas seulement des objets
- const store = createStore(rootReducer, applyMiddleware(thunk));
-
- /*
- const composeEnhancers = window.__REDUX_DEVTOOLS_EXTENSION_COMPOSE__ || compose;
-
- const store = createStore(
-   rootReducer,
-   composeEnhancers(applyMiddleware(thunk))
- );
- */
-
- // subscribe : cette fonction est appelée après chaque dispatch
- store.subscribe(() => {
-   console.log("Updated State:", store.getState());
- });
-
- export { store };
```

Étape 6 → 7 etape-6-premier-slice → etape-7-redux-toolkit

Les 3 slices (phone, tv, camera), extraReducers (promo TV), RTK Query (createApi)

MODIFIÉ `src/App.css` +0 −13

```
@@ -65,16 +65,3 @@
    font-weight: 600;
    color: white;
  }
-
- .quantity {
-   margin: 0;
- }
-
- .input {
-   width: 60px;
-   padding: 4px;
-   font-size: 16px;
-   border: 1px solid #ccc;
-   border-radius: 4px;
-   text-align: center;
- }
```

MODIFIÉ `src/App.jsx` +1 −0

```
@@ -2,6 +2,7 @@
  import Home from "./pages/Home.jsx";
  import { Provider } from "react-redux";
  import { store } from "./store/store.js";
+
  function App() {
    // Provider donne le store à tous les composants (useSelector, useDispatch)
    return (
```

AJOUTÉ `src/apis/index.js` +70 −0

```
@@ -1,0 +1,70 @@
+ // RTK Query (Redux Toolkit Query) : on crée une "API slice"
+ // Objectif : faire des appels HTTP + gérer cache / loading / error automatiquement
+
+ import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
+
+ /**
+  * commentsApi = "API slice"
+  * Elle contient :
+  * - un reducer (pour stocker cache + status dans Redux)
+  * - un middleware (pour déclencher/optimiser les requêtes)
+  * - des endpoints (GET/POST/...)
+  * - des hooks React auto-générés (useGetCommentsQuery, etc.)
+  */
+ export const commentsApi = createApi({
+   /**
+    * reducerPath = le nom de la clé dans le store Redux
+    * Exemple : state.commentsApi => contiendra le cache, loading, erreurs, etc.
+    *
+    * Dans le store on fera :
+    * reducer: { [commentsApi.reducerPath]: commentsApi.reducer }
+    */
+   reducerPath: "commentsApi",
+
+   /**
+    * baseQuery = le "client HTTP" utilisé par RTK Query
```

```
+    * fetchBaseQuery est une petite couche au-dessus de fetch (pas besoin d'axios)
+    *
+    * baseUrl = URL de base, les endpoints ajouteront juste le chemin
+    */
+   baseQuery: fetchBaseQuery({
+     baseUrl: "https://jsonplaceholder.typicode.com",
+   }),
+
+   /**
+    * endpoints = on déclare ici toutes les routes API
+    * builder permet de créer :
+    * - builder.query(...)  => GET (lecture) + cache
+    * - builder.mutation(...) => POST/PUT/DELETE (écriture)
+    */
+   endpoints: (builder) => ({
+     /**
+      * getComments = nom de l'endpoint
+      * builder.query => c'est un endpoint de lecture (GET)
+      */
+     getComments: builder.query({
+       /**
+        * query = fonction qui construit l'URL de la requête
+        * postId est un paramètre (optionnel) avec une valeur par défaut = 1
+        *
+        * Si postId = 1 => GET /comments?postId=1
+        * Si postId = 5 => GET /comments?postId=5
+        *
+        * RTK Query gère automatiquement :
+        * - loading / success / error
+        * - cache + dedupe (évite des appels identiques)
+        */
+       query: (postId = 1) => `/comments?postId=${postId}`,
+     }),
+   }),
+ });
+
+ /**
+  * RTK Query génère automatiquement des hooks React
+  * Le nom est basé sur l'endpoint :
+  * getComments => useGetCommentsQuery
+  *
+  * Utilisation :
+  * const { data, isLoading, error } = useGetCommentsQuery(1);
+  */
+ export const { useGetCommentsQuery } = commentsApi;
```

MODIFIÉ **src/pages/Home.jsx** +38 −11

```
@@ -2,39 +2,66 @@
  import tvImg from "../assets/tv.png";
  import cameraImg from "../assets/camera.png";

- // buyPhone vient du slice ; buyTv et buyCamera encore des anciennes actions
  import { buyPhone } from "../store/phoneSlice";
- import { buyTv, buyCamera } from "../store/actions";
- import { useDispatch, useSelector } from "react-redux";
+ import { buyTv } from "../store/tvSlice";
+ import { buyCamera } from "../store/cameraSlice";
+
+ import { useSelector, useDispatch } from "react-redux";
+ import { useGetCommentsQuery } from "../apis";

  function Home() {
    // useDispatch envoie les actions ; useSelector lit le state (re-render quand il change)
    const dispatch = useDispatch();
```

```
13  15      const phone = useSelector((state) => state.phone);
    16  +   const camera = useSelector((state) => state.camera);
14  17      const tv = useSelector((state) => state.tv);
15      -   const camera = useSelector((state) => state.camera);
    18  +   // Hook généré par RTK Query : il fait la requête et donne data / isLoading / error
    19  +   const { data, isLoading, error } = useGetCommentsQuery(1);
    20  +   console.log({ data, isLoading, error });
    21  +
16  22      return (
17  23        <div>
18  24          <h2>{"Welcome to our Application"}</h2>
19  25          <div className="container">
20  26            <div className="item">
21  27              <img src={phoneImg} alt="" width={100} height={100} />
22      -           <div className="availability">{phone.total} available</div>
23      -           <button className="button" onClick={() => dispatch(buyPhone(2))}>
    28  +           <div className="availability" id="phones">
    29  +             {phone.total} available
    30  +           </div>
    31  +           <button
    32  +             className="button"
    33  +             onClick={() => {
    34  +               dispatch(buyPhone());
    35  +             }}
    36  +           >
24  37                Buy
25  38              </button>
26  39            </div>
27  40            <div className="item">
28  41              <img src={tvImg} alt="" width={100} height={100} />
29      -           <div className="availability">{tv.total} available</div>
30      -           <button className="button" onClick={() => dispatch(buyTv())}>
    42  +           <div className="availability" id="tvs">
    43  +             {tv.total} available
    44  +           </div>
    45  +           <button
    46  +             className="button"
    47  +             onClick={() => {
    48  +               dispatch(buyTv());
    49  +             }}
    50  +           >
31  51                Buy
32  52              </button>
33      -         </div>{" "}
    53  +         </div>
34  54            <div className="item">
35  55              <img src={cameraImg} alt="" width={100} height={100} />
36      -           <div className="availability">{camera.total} available</div>
37      -           <button className="button" onClick={() => dispatch(buyCamera())}>
    56  +           <div className="availability" id="cameras">
    57  +             {camera.total} available
    58  +           </div>
    59  +           <button
    60  +             className="button"
    61  +             onClick={() => {
    62  +               dispatch(buyCamera());
    63  +             }}
    64  +           >
38  65                Buy
39  66              </button>
40  67            </div>
```

SUPPRIMÉ **src/store/actions.js** +0 −17

```
@@ -1,17 +1,0 @@
```

```
- const BUY_TV = "BUY_TV";
-
- const buyTv = () => {
-   return {
-     type: BUY_TV,
-   };
- };
-
- const BUY_CAMERA = "BUY_CAMERA";
-
- const buyCamera = () => {
-   return {
-     type: BUY_CAMERA,
-   };
- };
-
- export { BUY_TV, buyTv, BUY_CAMERA, buyCamera };
```

AJOUTÉ **src/store/cameraSlice.js** +17 −0

```
@@ -1,0 +1,17 @@
+ // features/stock/cameraSlice.ts
+ import { createSlice } from "@reduxjs/toolkit";
+
+ const initialState = { total: 8 };
+
+ const cameraSlice = createSlice({
+   name: "camera",
+   initialState,
+   reducers: {
+     buyCamera(state) {
+       state.total -= 1;
+     },
+   },
+ });
+
+ export const { buyCamera } = cameraSlice.actions;
+ export default cameraSlice.reducer;
```

MODIFIÉ **src/store/phoneSlice.js** +23 −6

```
@@ -1,17 +1,34 @@
+ // features/stock/phoneSlice.ts
  import { createSlice } from "@reduxjs/toolkit";
+ // import { buyTv, buyTvPromotion } from "./tvSlice";
+
+ const initialState = { total: 5 };

  const phoneSlice = createSlice({
    name: "phone",
-   initialState: {
-     total: 5,
-   },
+   initialState,
    // Chaque reducer devient une action (phoneSlice.actions) ; Immer permet de « modifier » state directement
    reducers: {
-     buyPhone: (state, action) => {
-       state.total -= action.payload || 1;
+     buyPhone(state) {
+       // action.payload can be used for more complex logic
+       state.total -= 1;
      },
    },
+   // extraReducers (en commentaire) : réagir à une action d'un autre slice, ex. la promo TV
```

```
+   /*extraReducers: (builder) => {
+     builder.addCase(buyTv, (state) => {
+       state.total -= 1; // si achat TV => +1 phone
+     });
+   },*/
+   /*
+     extraReducers: (builder) => {
+     builder.addCase(buyTvPromotion, (state, action) => {
+       if (action.payload === "2025XMAS") {
+         state.total -= 1;
+       }
+     });
+   }, */
  });

  export const { buyPhone } = phoneSlice.actions;
- export const phoneReducer = phoneSlice.reducer;
+ export default phoneSlice.reducer;
```

SUPPRIMÉ **src/store/reducers.js** +0 −36

```
@@ -1,36 +1,0 @@
- // phone est passé dans phoneSlice : il reste tv et camera (passés en slice à l'étape 7)
- import { BUY_TV, BUY_CAMERA } from "./actions";
-
- const tvInitialState = {
-   total: 3,
- };
-
- const tvReducer = (state = tvInitialState, action) => {
-   switch (action.type) {
-     case BUY_TV:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
-     default:
-       return state;
-   }
- };
-
- const cameraInitialState = {
-   total: 8,
- };
-
- const cameraReducer = (state = cameraInitialState, action) => {
-   switch (action.type) {
-     case BUY_CAMERA:
-       return {
-         ...state,
-         total: state.total - 1,
-       };
-     default:
-       return state;
-   }
- };
-
- export { tvReducer, cameraReducer };
```

MODIFIÉ **src/store/store.js** +10 −4

```
@@ -1,13 +1,19 @@
+ // store.ts
  import { configureStore } from "@reduxjs/toolkit";
- import { phoneReducer } from "./phoneSlice";
```

```
3     - import { tvReducer, cameraReducer } from "./reducers";
      3 + import phoneReducer from "./phoneSlice";
      4 + import tvReducer from "./tvSlice";
      5 + import cameraReducer from "./cameraSlice";
      6 + import { commentsApi } from "../apis";
4     7
5     - // configureStore remplace createStore + combineReducers : on peut mélanger un slice et les anciens reducers
6     8   export const store = configureStore({
7     9     reducer: {
8     10      phone: phoneReducer,
9     11      tv: tvReducer,
10    12      camera: cameraReducer,
      13 +    commentsApi: commentsApi.reducer,
11    14    },
12    -   devTools: true,
      15 +  // On ajoute le middleware de RTK Query (requêtes, cache) à ceux par défaut
      16 +  middleware: (getDefaultMiddleware) =>
      17 +    getDefaultMiddleware().concat(commentsApi.middleware),
      18 +  devTools: true, // optionnel (par défaut true en dev)
13    19  });
```

AJOUTÉ **src/store/tvSlice.js** +20 −0

```
@@ -1,0 +1,20 @@
1  + // features/stock/tvSlice.ts
2  + import { createSlice } from "@reduxjs/toolkit";
3  +
4  + const initialState = { total: 3 };
5  +
6  + const tvSlice = createSlice({
7  +   name: "tv",
8  +   initialState,
9  +   reducers: {
10 +     buyTv(state) {
11 +       state.total -= 1;
12 +     },
13 +     /*buyTvPromotion(state, action) {
14 +       state.total -= 1;
15 +     }/*, */
16 +   },
17 + });
18 +
19 + export const { buyTv } = tvSlice.actions;
20 + export default tvSlice.reducer;
```